

Service-Oriented Architecture (SOA) – Simplified Points

Basic Meaning

- SOA is a software development method that builds applications using independent components called services.
- Each service performs a specific business capability.
- Services can communicate with each other even if they use different platforms or programming languages.
- SOA is both a specification and a methodology for building distributed, platform-independent services.

How Services Work

- A service is a repeatable task within a business process.
- A business task is formed by combining multiple services.
- SOA uses message passing between components to provide services when requested.
- Clients send a message (with metadata) in a standard format to access a service.
- The service processes the message and returns a response.
- A common example of a message format is XML sent using SOAP.

Architecture Characteristics

- There are no direct API links; only messages are exchanged.
- The client checks whether the returned result is appropriate.
- SOA is viewed as linking several services to form an integrated business process.
- A component that provides a service is exposed as an “endpoint.”

Common Message Formats and Languages

- XML + SOAP is most commonly used.
- Other formats include WSDL, WSS, and WS-BPEL.
- WSDL describes the service interface, binding details, and the nature of the service/endpoint.
- SCDL defines the internal service component details not included in WSDL.

Role of Middleware

- Service providers and consumers usually do not communicate directly.
- Middleware acts as a transaction manager and translator.
- Middleware can discover and list available services through a registry.
- Security and trust services are built into middleware.

- Business logic can also reside inside middleware.
- UDDI protocol is commonly used for broadcasting and discovering Web services, often using ebXML documents.
- Service consumers locate a service in the registry and bind their requests to it.

Core SOA Values

- Business value
- Strategic goals
- Intrinsic interoperability
- Shared services
- Flexibility
- Evolutionary refinement

SOA Protocol Stack

- SOA uses multiple protocols that handle different required functions.
- Other supported service technologies include CORBA, REST, RPC, DCOM, Jini, DDS, WCF, etc.
- SOA is flexible and neutral, making it useful for designing complex applications.
- The message provides data, the service responds, and the client verifies correctness.
- The formal definition of a transaction in SOA is called a “contract.”
- SOA uses an orchestrator or broker to ensure correct message transactions.
- SOA focuses only on the interaction boundary between the client (consumer) and the service provider.

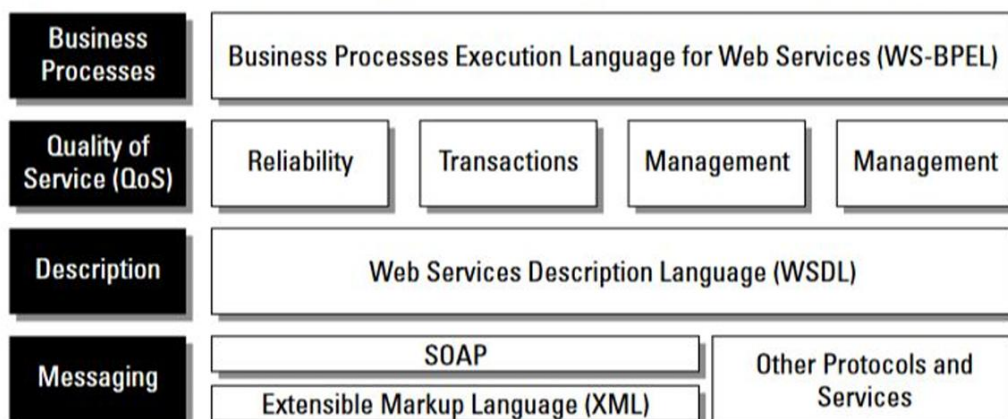


RV College of
Engineering®

SOA-Protocol Stack

Go, change the world®

A protocol stack for SOA showing the relationship of each protocol to its function



SOA – Choreography

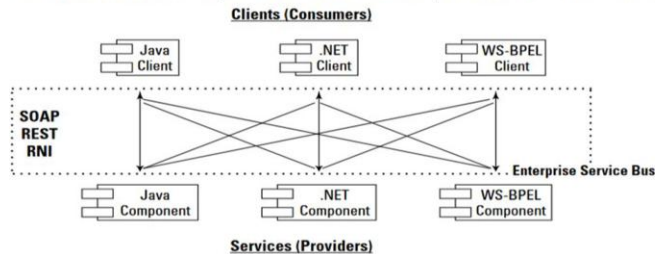


RV College of
Engineering®

SOA-Client Construction

Go, change the world

SOA allows for different component and client construction, as well as access to each using different protocols.

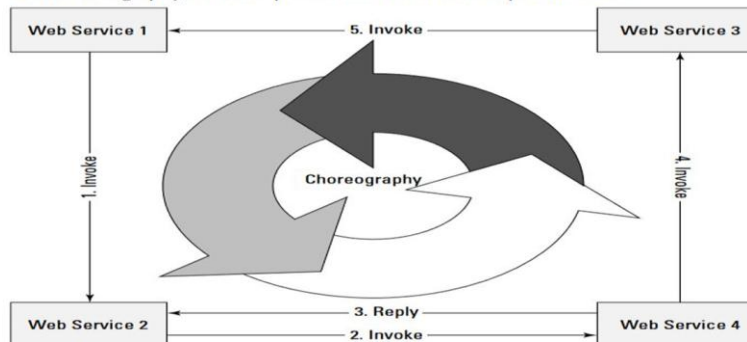


RV College of
Engineering®

SOA- Choreography

Go, change the world

With choreography, business process execution is a cooperative affair.



- Components of different types can communicate using different protocols in SOA.
- When Web services are combined to form business processes, the integration must be managed.
- Two methods are used to combine Web services: **orchestration** and **choreography**.
- In **orchestration**, a middleware service centrally coordinates all Web service operations.
- All services send and receive messages through this central orchestrator.
- In **choreography**, there is **no central controller**.
- Each Web service knows:
 - When to process a message
 - Which client or component to interact with
- Choreography is collaborative; each service handles:
 - Which operations to run and when
 - How messages are structured
 - Message timing and related decisions

Orchestration

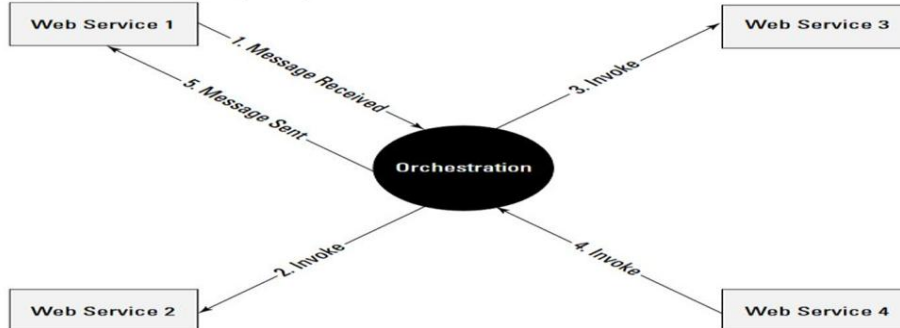


RV College of
Engineering®

Orchestration

Go, ch

An orchestrated business process uses a central controlling service or element, referred to as the conductor, or less frequently, the coordinator.



SOA – Choreography (continued)

- Most mature SOA implementations prefer **orchestration** over choreography.
- Orchestration has a single central service that manages all processes, making updates easier.
- Integrating Web services becomes simpler because services do not need knowledge of the business process.
- Centralizing business logic makes error handling easier.
- It also helps in managing and analyzing events that occur outside the business process.
- Event handling is part of **event-driven SOA (SOA 2.0)**, which includes both random and scheduled events triggered inside or outside the business process.
- One method of implementing orchestration is through an **Enterprise Service Bus (ESB)**.
- An ESB provides a middleware layer for event management with a messaging infrastructure.

Event-driven SOA or SOA 2.0

- Event-driven SOA (SOA 2.0) extends traditional SOA to respond to events that occur during business processes or influence those processes.
- Example: If a system detects a rapid increase in sales on a website, it can analyze what events might be influencing customer buying decisions.
- SOA 2.0 handles such analysis by allowing:
 - Low-level events to trigger a business process
 - Events to be correlated with SOA data
 - A business process to stop if expected events do not occur

- Automatic reactions or responses based on event triggers
 - To support this, a **Causal Vector Engine (CVE)** with built-in artificial intelligence is added to the SOA design.
 - Events are analyzed based on:
 - Event sequences
 - Event relationships
 - Event timing
 - The CVE reacts to conditions using predefined rules.
 - CVE systems often display events in different contexts on a console for observers to analyze and take action.
-

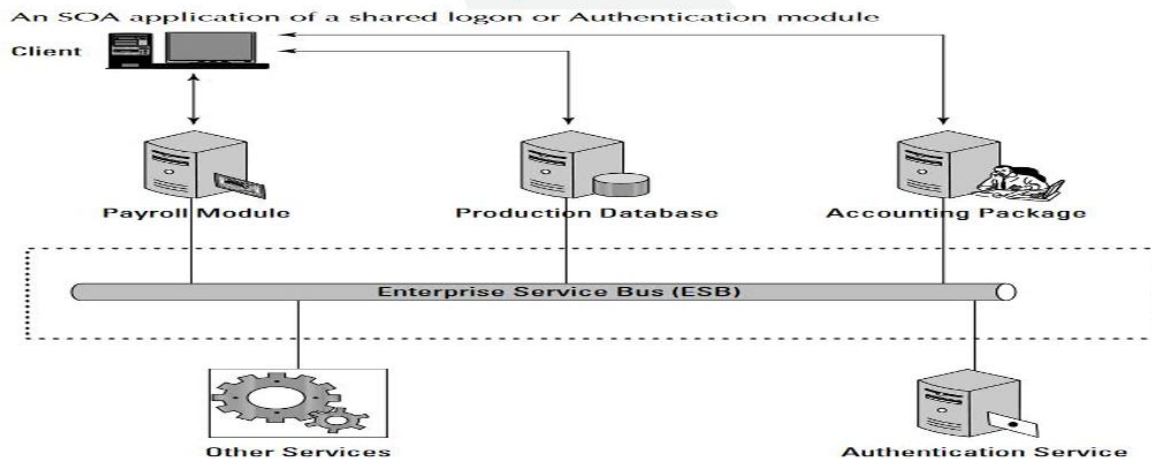
Event-driven SOA or SOA 2.0 (continued)

- CVE applications may allow querying event data, similar to how stock trading systems query trading data.
 - CVE provides heartbeat and correlation functions similar to stock market applications.
 - For the client (service consumer):
 - They only need to know how to call the service and interpret the returned results.
 - They do not know how the component processes the request, where it runs, or which systems are involved.
 - The client must validate whether the returned result meets expectations.
 - The SOA component behaves as a **black box** to the client.
 - SOA requires only that components follow standard endpoint rules.
-

Event-driven SOA or SOA 2.0 (continued)

- This abstraction allows service providers to upgrade, replace, or relocate components without affecting clients.
- Quality of Service (QoS) can still be measured and maintained.
- SOA provides rules that let applications access authentication modules in their own way.
- Benefits of SOA:
 - Adds powerful capabilities with much less cost and effort
 - Allows applications to be federated if needed
- Limitation of SOA:
 - You cannot deeply customize a service if it is provided by a third party

or which is that transactions succeed or they fail and are rolled back.



The Enterprise Service Bus

- An ESB is not a physical network bus; it is an architectural pattern made of network services that manage transactions in SOA.
- It separates clients from components on a transactional basis, giving the system a loosely coupled structure.
- Messages flow from client to component through the ESB, which manages the transaction regardless of where the services are located.
- An ESB is useful but not mandatory in SOA because business processes may involve many messages and events, and distributed processing can be unreliable.
- The ESB acts as a transaction broker, ensuring messages reach their correct destination and are handled properly.

The Enterprise Service Bus

- The ESB performs mediation functions such as:
 - Message translation
 - Registration
 - Routing
 - Logging
 - Auditing
 - Maintaining transactional integrity
- Transactional integrity is similar to ACID in databases—transactions must either fully succeed or fail and roll back.

The Enterprise Service Bus

- An ESB can be part of a network operating system or implemented using middleware products.
 - It creates a virtual environment on top of an enterprise messaging system where services are advertised and accessed.
 - An ESB is essentially a message transaction system.
 - Example: IBM WebSphere ESB 7.0 uses open standards such as Java EE, EJB, WS-Addressing, WS-Policy, and Kerberos.
 - It runs on WebSphere Application Server and is compatible with Open SCA.
 - It includes a Service Federation Management tool and an integrated Registry and Repository.
-

The Enterprise Service Bus

Typical features found in ESBs include:

- Monitoring services for managing events.
 - Process management services to manage message transactions.
 - Data repositories or registries to store business logic and support governance.
 - Data services to pass messages between clients and services.
 - Data abstraction services to translate message formats as needed.
 - Governance services to ensure compliance with government regulations.
 - Security services to validate clients and services and allow secure message exchange.
-

The Enterprise Service Bus

- A **repository** and a **registry** are both data stores but serve different roles in SOA.
- A repository stores references to SOA components, their source code, and linking information used to provide services.
- A registry stores the rules, descriptions, and definitions of services—i.e., the metadata.
- A repository is similar to a name server, while a registry functions like a directory service (domain).
- The service broker uses rules in the registry to translate and deliver messages.
- The registry is the central location where developers store component descriptions and publish services.
- SOA services also include provider interfaces and standard network protocols.
- Developers may create a Business Process Orchestration module to coordinate access and maintain transactional integrity across multiple business applications.

Service Catalogs

- In a large SOA system, finding a specific service and its requirements can create heavy network overhead.
 - To simplify this, SOA often includes a **catalog service**.
 - A catalog service stores information such as:
 - What services are available (internal and external)
 - How to use each service
 - Which applications depend on a particular service
 - How services are related to one another
 - Who owns the service and how it can be modified
 - The event history of a service (service levels, outages, etc.)
 - The nature of service contracts
-

Service Catalogs

- Service catalogs are dynamic and constantly updated.
- Catalog servers may have the following forms:
 - A standalone catalog server for a single site
 - A global catalog service created by merging multiple catalog servers across sites (requires synchronization to keep data unified)
 - A federated catalog service, where multiple global catalog servers share information through a trusted query relationship
- Catalog services significantly affect performance in large systems.
- As a SOA internetwork grows, catalog services become essential.
- An internetwork is created by combining multiple separate networks, similar to how the Internet was formed.

SOA Communications

- Message passing in SOA uses two protocol types:
 - A **data interchange format**
 - A **network protocol** that carries the message
 - A client connected to an ESB communicates using protocols like **HTTP**, **REST**, or **JMS**.
 - Messages are usually formatted in **XML** or **SOAP**.
 - SOAP uses XML for messages and relies on protocols like **HTTP** and **RPC** for sending and negotiating messages.
 - Clients and services can be written in Java, .NET, WS-BPEL, or other programming languages.
 - The key requirement is the ability to **transport and translate messages** so both sides understand them.
-

SOA Communications

- An ESB may support multiple protocol and format combinations.
 - Examples from WebSphere ESB include:
 - XML/JMS
 - SOAP/JMS
 - SOAP/HTTP
 - Text/JMS
 - Bytes/JMS
 - **WSDL** is a widely used XML protocol for messaging in Web services and SOA.
-

SOA Communications

- WSDL 1.1 is a W3C standard; WSDL 2.0 is newer but not yet ratified.
- WSDL 2.0 has better support for **RESTful applications**, but tool support is still limited.
- The most common transport for WSDL is **SOAP**.
- WSDL files contain XML data and an XML schema.
- REST provides different capabilities than SOAP:
 - Each URL represents an object you can access or modify.
 - Uses standard HTTP commands: **GET**, **POST**, **PUT**, **DELETE**.
- SOAP exposes Web objects through an API and transfers data using XML.
- REST is lightweight, easier to implement, and has less overhead than SOAP.

SOA Communications

- SOAP offers more precision and a more error-free consumption model, along with advanced development tools.
- Most major Web services use REST, but many also combine REST and SOAP for combined benefits.

WSDL key objects for message transfer:

- **Service object** – container where the service resides.
- **Port (endpoint)** – the unique address of the service.
- **Binding** – defines the interface (e.g., RPC) and transport (e.g., SOAP).
- **PortType** – defines what the Web service can do, including operations and required messages.
- **Operation** – the action performed on the message.
- **Message content** – the data and metadata the operation works on (may have multiple parts, each with type info).
- **Types** – data definitions, usually in the XML schema included in the WSDL.

SOA Communications

- The XML document in WSDL sets up the namespace, defines the interface, specifies the binding, names the service, adds documentation, and provides a schema for validation.
- Message types are declared in the message descriptions.
- XML schema may be in separate files but is often included inline in WSDL.
- A WSDL file contains message data for a transaction but does not capture the full SOA design.
- Additional requirements are expressed through a **service contract**.
- A service contract defines:
 - The data to be processed
 - The metadata
 - The intended service
 - How the service will act on the message

SOA Communications

- **Header:** Includes service name, version, owner, and responsibilities (often defined using a RACI matrix).
 - **Service Type:** Examples include data, business, integration, presentation, and process services.
 - **Functional Specification:** Defines functional requirements, operations, methods, invocation details, URL, and interface nature.
 - **Transaction Attributes:** Describe how a transaction is managed, tracked, QoS level, SLA requirements, and security parameters.
 - These attributes also specify the message's role in a process and the semantics of interaction.
-
-

Business Process Execution Language (BPEL) – 10-Point Summary

1. BPEL handles groups of atomic transactions to execute complete business processes.
2. It is a standard Web service execution language maintained by OASIS.
3. BPEL is a meta-language containing executable commands and abstract business logic.
4. It is used to compose, orchestrate, and coordinate business processes in SOA.
5. BPEL supports asynchronous communication between services.
6. It uses XML and works with SOAP, WSDL, UDDI, WS-ReliableMessaging, WS-Addressing, WS-Coordination, and WS-Transactions.
7. BPEL combines IBM's WSFL and Microsoft's XLANG concepts, forming BPEL4WS.
8. BPEL4People extends BPEL to support human interaction tasks.
9. Business processes in BPEL have both internal (executable) and external (abstract) views.
10. BPEL provides control flow, logic, branching, process orchestration, and error handling for long-lived, asynchronous transactions.

Business Process Modeling – Concise Summary

Unified Modeling Language (UML)

- UML (by OMG) provides graphical diagrams to model software systems.
- Includes actors, business processes, components, routines, database schemas, and activities.
- Consists of **7 structural** and **4 behavioral** diagram types.
- Widely used in industry and can be reduced to executable code.

- UML uses **XMI** (XML Metadata Interchange) to exchange metadata in XML.

XMI and SysML

- XMI is an OMG and ISO standard for exchanging metadata; files are not always interchangeable across tools.
- **SysML** is an open-source, simplified extension of UML supporting 7 of UML 2.0's diagrams.
- SysML uses XMI and is developing support for ISO 10303 STEP AP-233.

Business Process Modeling Notation (BPMN)

- BPMN represents business processes through visual workflow diagrams (BPD).
 - Created by BPMI, now part of OMG.
 - BPMN diagrams can be converted into WS-BPEL for execution by Web services.
-

Service-Oriented Modeling Framework (SOMF)

- SOMF models SOA components visually as objects and relationships.
- Helps plan, optimize, trace message paths, and test “what-if” scenarios.
- SOMA (related approach) breaks systems into:
 - **Services**
 - **Service components**
 - **Information flows**
- Identifies domains, functional groups, process variables, and service granularity.

SOMF Service Life Cycle Stages

1. Conceptualization – define supported processes
 2. Discovery – determine how components are exposed
 3. Analysis – identify service–client relationships
 4. Integration – implement messaging infrastructure
 5. Logical design – build system logic, orchestration/choreography
 6. Architecture design – design components and interfaces
 7. System implementation – build and test components
-

SOMF Message-Passing Topologies

- **Circular topology:**
 - No orchestrator; components decide what message to process and where to send next (choreography-based).

- **Hierarchical topology:**
 - Services arranged in parent–child tree; messages travel up and down branches.
 - Clear structure but less optimized message passing.
 - **Network topology:**
 - Many-to-many communication; very fast message passing but requires many connections.
 - **Star topology:**
 - Central hub service; ideal for orchestration, broadcasting, and publish/subscribe systems.
-

Service Types in SOMF

- **Atomic service:**
 - Smallest usable unit; cannot be decomposed.
 - Example: customer lookup (ID, name, address).
- **Composite service:**
 - Collection of related services (e.g., bank account or ESB functions).
 - Often hierarchical and coarse-grained.
- **Service cluster:**
 - Group of composite and/or atomic services forming a larger process module (e.g., payroll module).

How SOMF Models Services

- Services have distinct shapes and can be:
 - Aggregated
 - Decomposed
 - Unified
 - Intersected
 - Subtracted
- Connections show relationships such as generalization or specialization.
- Final model is optimized and mapped to system implementation; some tools generate executable code.

Managing and Monitoring SOA – Concise Summary

- Monitoring and managing SOA is critical in large deployments because SOA complexity requires **proactive oversight**, not reactive troubleshooting.
- SOA management tools run continuously and are multifaceted, handling monitoring, mapping, configuration, and optimization.

Major SOA Management Tools

- **HP OpenView SOA Manager** – provides dynamic mapping, monitoring, and optimization of Web services, software assets, and virtual services.
- **IBM Tivoli ITCAM** – focuses on change management, SOA lifecycle management, and integrates with WebSphere and other Tivoli systems.
- **Oracle BPEL Process Manager** – manages BPEL workflows and supports ESB creation.

Industry Trends & Additional Tools

- SOA management tools evolve rapidly; many small vendors' products are acquired and merged into larger suites (e.g., Oracle acquiring AmberPoint).
- **BMC AppSight** and **Tidal Intersperse** provide automated problem resolution and root-cause analysis.
- **CA Wily SOA Solution** maps SOA transactions, dependencies, ESBs, portals, and Web services.
- **iTKO LISA** specializes in testing Web service components.
- **OpTier CoreFirst** is another SOA transaction manager.

Challenges in SOA Management

- SOA infrastructures are **highly distributed**, requiring strong discovery mechanisms.
- They are also **highly virtualized**, with workloads moving across virtual and physical servers dynamically.
- This constant shifting increases complexity, making configuration and change management difficult.
- Virtualization will remain a major challenge for SOA management tools in the future.

SOA Security – Concise Summary

- SOA sends large volumes of messages across networks, making it vulnerable to traditional attacks like hijacking, spoofing, redirection, and blocking.
- Because SOA removes traditional application boundaries, **application-level security alone is insufficient**.

- Cisco's **Application Oriented Networking (AON)** and Citrix's **NetScaler** provide policy-based XML message security.

OASIS Security Standards for SOA

1. SAML (Security Assertion Markup Language)

- XML-based standard for authentication and authorization between client and service.
- Used in Single Sign-On (SSO) to allow users to access distributed SOA resources after one login.

2. WS-Security (WSS)

- Adds security tokens (Kerberos, SAML, X.509) to messages.
- Uses XML Signature and XML Encryption for secure client/service communication.

3. WS-SecureConversation

- Creates and shares a **security context** for ongoing communication.
- Works with WS-Security, WS-Trust, and WS-Policy to attach security context tokens to SOAP messages.

4. WS-SecurityPolicy

- Defines policies requiring messages to be signed and encrypted.
- Extends WS-Security, WS-Trust, and WS-SecureConversation.

5. WS-Trust

- Extends WS-Security to issue, renew, and validate security tokens.
- Uses a **Security Token Service (STS)** for token generation and key exchange.

XML Gateways & PKI in SOA Security

- XML gateways inspect SOAP/REST XML messages, verify their source, and ensure secure delivery.
- Requires **Public Key Infrastructure (PKI)** to enforce encryption and digital signatures.

SOA Security Tools

- Progress Software's **Actional 8.0** (includes Mindreef SOAPscope) provides XML middleware for:
 - Diagnostic testing
 - Web service governance
 - Monitoring XML appliances and application servers

The Open Cloud Consortium (OCC) – Concise Summary

- The Open Cloud Consortium (OCC) is a group of universities and companies that develops **cloud computing standards** and supports **interoperability** between different cloud frameworks.

Key Functions of OCC Working Groups

- Develop **benchmarks** for cloud performance measurement.
 - Their benchmark for large data clouds is **MalStone**, with a data generator called *malgen*.
- Provide **testbeds** for vendors to test cloud applications, including:
 - The **Open Cloud Testbed**
 - The **Intercloud Testbed**
- Support creation of **open-source reference implementations** for cloud computing.
- Extend architectures for large data clouds through:
 - Distributed file systems
 - Table services
 - MapReduce-based computing (similar to Google's model)

Related Technologies

- **MapReduce** is Google's framework for processing large distributed datasets using the Google File System (GFS).
- **Apache Hadoop** is an open-source implementation inspired by MapReduce and GFS.

Additional OCC Initiatives

- Manage cloud infrastructure for scientific research through the **Open Science Data Cloud (OSDCP)** Working Group.

Relating SOA and Cloud Computing – Concise Summary

- Cloud computing is still developing, and while Web services *can* use SOA, it is **not required** for cloud applications.
- Many early cloud applications do not need the flexibility and loose coupling that SOA provides.
- As cloud applications grow in diversity and complexity, **SOA becomes useful as a standardized, loosely coupled architectural blueprint** for accessing different optimized services.

- SOA is loosely coupled because **services are independent from the messaging system**, making components easy to replace with minimal or no programming.
- Developers may lose some customization options, but gain easier application design and maintenance.
- Although SOA-based applications may seem tightly integrated, the underlying architecture remains loosely coupled.